

Technical Report

Design and Verification of a Structural 4-bit Kogge-Stone Parallel Prefix Adder

Reece M. Dobro

May 25, 2026

Abstract

The process of developing and testing a 4-bit Kogge-Stone adder in the Verilog programming language using AMD Vivado is discussed in this report. The 4-bit adder was programmed through dataflow modeling and structural-level implementation of the propagate and generate logic. Testing and validation consisted of four 4-bit values with reasoning for the choices included in this report. Successful addition of all 4 test values demonstrated the 4-bit adder's reliability in both its programming and structure.

1 Introduction, Background, and Theory

The main objective of this project is to program and test a 4-bit Kogge-Stone adder in Verilog through structural-level descriptions. A Kogge-Stone adder is a type of binary adder that is able to evaluate carry values simultaneously, allowing for a faster calculation compared to a ripple carry adder. The Kogge-Stone adder does this by finding carry and propagate values for each stage of calculation. The number of stages is equal to $\log_2(n) + 1$, where n is the number of bits being added, with 3 stages needed for a 4-bit adder. The first stage is the initial finding of propagate and carry values, which doesn't require previous values unlike the other stages. Once each stage is evaluated, a 4-bit sum and 1-bit carry-out value can be found.

The structural code for the Kogge-Stone adder is contained in a single module, *ksa_4bit_df*. Figure 1 shows a block diagram of the module. As shown, the module has 2 inputs: *A*, a 4-bit vector and *B*, a 4-bit vector. The outputs of the module are *S*, which is a 4-bit vector that holds the sum of $A+B$ and *Cout*, a 1-bit value that holds the carry-out value of $A+B$. This carry-out will be held high any time the total value of $A+B$ exceeds 15. The nets of the module are all 4-bit vectors, with 3 of them being used for holding propagate values for each of the 3 stages and the other 3 being used to hold generate values for each of the 3 stages. They are all 4-bit vectors as the output, *S* is a 4-bit vector. The functionality of the module is shown in Figure 2. It contains the processing for propagate and generate values for each of the 3 stages as well as for the sum and carry out. Stage 1 is the simplest as it doesn't refer to previous stages to find propagate and generate values. For the sum calculation, each sum bit is found by an XOR operation with the initial propagate value for the column and the carry of the previous bit. However, as the LSB is the lowest bit, it doesn't have a value for the carry out of its previous bit, rather, the carry-in value is used. Since there is no carry-in value for this module, it will always be low.

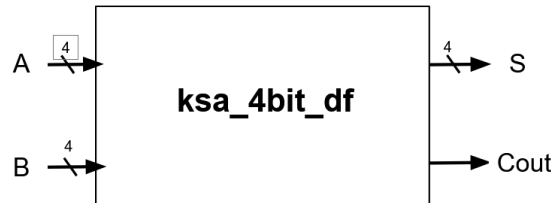


Figure 1: Block Diagram of module *ksa_4bit_df*

```

assign P0=A^B;
assign G0=A&B;
//stage 2
assign P1[0]=P0[0];
assign P1[1]=P0[0]&P0[1];
assign P1[2]=P0[2]&P0[1];
assign P1[3]=P0[3]&P0[2];
assign G1[0]=G0[0];
assign G1[1]=(P0[1]&G0[0])|G0[1];
assign G1[2]=(P0[2]&G0[1])|G0[2];
assign G1[3]=(P0[3]&G0[2])|G0[3];
//stage 3
assign P2[0]=P1[0];
assign P2[1]=P1[1];
assign P2[2]=P1[2]&P1[1];
assign P2[3]=P1[3]&P1[2];
assign G2[0]=G1[0];
assign G2[1]=G1[1];
assign G2[2]=(P1[2]&G1[0])|G1[2];
assign G2[3]=(P1[3]&G1[1])|G1[3];
//sum
assign S[0]=P0[0]^1'b0;
assign S[1]=P0[1]^G2[0];
assign S[2]=P0[2]^G2[1];
assign S[3]=P0[3]^G2[2];
//carryout
assign Cout=G2[3];

```

Figure 2: Functionality for module *ksa_4bit_df*

The testbench for this project, *ksa_4bit_df_tb.v*, serves the purpose of testing the module *ksa_4bit_df* with 4 predetermined sets of test values for the 4-bit input vectors *A* and *B*. The code for the testbench is shown in Figure 3. In addition to assigning test vectors to the inputs so that they can be seen through waveforms, it also displays the inputs and outputs in the TCL Console in unsigned binary. This allows for the test cases to be easily analyzed and recorded.

```

`timescale 1ns / 1ps
module ksa_4bit_df_tb;
reg [3:0] A;
reg [3:0] B;
wire [3:0] S;
wire Cout;
ksa_4bit_df dut(A,B,S,Cout);

initial
begin
    A=4'b0000;
    B=4'b0000;
    #250
    $display("A:%b B:%b | Sum: %b Cout: %b",A,B,S,Cout);
    A=4'b0000;
    B=4'b0001;
    #250
    $display("A:%b B:%b | Sum: %b Cout: %b",A,B,S,Cout);
    A=4'b1010;
    B=4'b1101;
    #250$display("A:%b B:%b | Sum: %b Cout: %b",A,B,S,Cout);
    A=4'b0001;
    B=4'b1111;
    #250
    $display("A:%b B:%b | Sum: %b Cout: %b",A,B,S,Cout);

end
endmodule

```

Figure 3: Full Code for module *ksa_4bit_df_tb*

Figure 4 shows the overall file structure for this project. The image is taken from the *Sources* panel in AMD Vivado and is setup to have the testbench module and adder module in separate files, as per project specifications. As shown in Figure 4, no constraints were used as no FPGA was used for this project. Due to the relative simplicity of this project, no utility sources were used.

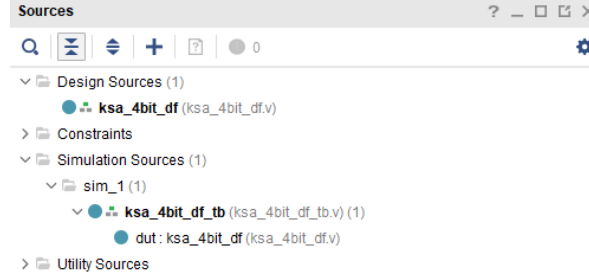


Figure 4: File Structure in AMD Vivado

1.1 Test Values and Justification

In order to confirm the success of the Kogge-Stone adder module, four sets of test values for a and b were given. These values and their expected outputs are shown in Table 1. If the module's outputs matched the correct outputs, then the structure and logic of the Kogge-Stone adder and the code written for it will be shown to be successful.

Inputs		Expected Output		Justification
a (4-bit)	b (4-bit)	Sum (4-bit)	C_{out}	
0000	0000	0000	0	Verifies the identity property, $a + 0 = a$
0001	1111	0000	1	Tests carry propagation and overflow behavior
0000	0001	0001	0	Confirms correct addition without carry
1111	1111	1110	1	Tests maximum input values and full carry-out

Table 1: 4-bit Kogge-Stone Adder Test Values and Justification

2 Results and Analysis

The output of the Kogge-Stone adder, (the 4-bit vector *sum* and the 1-bit port *carry-out*) are shown through Figures 5 and 6. Figure 6 shows the waveform output from AMD Vivado of the testbench and Figure 5 shows the output of the testbench through the TCL Console. These figures provide two different views of the outputs. Figure 6, the output of the waveforms, demonstrates how the changes in the test values cause a change in the *sum* and *carry-out* values. Figure 5 allows for the raw outputs and test values to be analyzed and compared to the expected outputs of the program. The data generated from Figure 5 was compiled into Table 2, along with the values from Table 1, for simplicity and comparison.

```
A: 0000 B: 0000 Sum: 0000 Cout: 0
A: 0001 B: 1111 Sum: 0000 Cout: 1
A: 0000 B: 0001 Sum: 0001 Cout: 0
A: 1111 B: 1111 Sum: 1110 Cout: 1
```

Figure 5: TCL Console Output of the Testbench



Figure 6: Waveforms Output of the Testbench

As shown by Table 2, the output of the Kogge-Stone adder matches the expected output. This successful verification demonstrates that the adder is properly implemented and functions as intended. By passing the given test values, the adder is shown to correctly handle edge cases, carry propagation, and maximum input conditions.

Inputs		Program Output		Expected Output	
a (4-bit vector)	b (4-bit vector)	Sum	C_{out}	Sum	C_{out}
0000	0000	0000	0	0000	0
0001	1111	0000	1	0000	1
0000	0001	0001	0	0001	0
1111	1111	1110	1	1110	1

Table 2: 4-bit Kogge-Stone Adder Test Value Verification

A potential weak point in the data is that only 4 test cases are used. Since there are 2^4 possible combinations for each of the 4-bit vectors a and b , there are 2^8 possible test values, with the four sets chosen covering only around 1.5% of the total possible test values. However, it would be impractical to use all possible test values to test out the completed module due to the length of time needed to manually verify and document each output.

As the outputs from the module matched the expected results, the Verilog program and its logical structure are deemed successful.

3 Conclusions

The correct implementation and verification of a 4-bit Kogge-Stone adder using structural modeling in Verilog were achieved in this project. A correct interpretation of a 4-bit Kogge Stone adder constructed using data-flow modeling is also demonstrated. However, while the program was successful, there are areas that could be improved for future design iterations. As seen in Figure 2, the naming convention for the propagate and carry values was $Pm[n]$, where m is the stage and n is the sum bit with 0 being the LSB. This could be made into a 2-D array, which would make the code more readable and scalable. If a larger bit adder was to be created using Kogge-Stone, then significantly more propagate and carry arrays would needed to be made. This would not be the case if a 2-D array could be used, as a bus could be used. Additionally, the test cases focused solely on functional correctness. Time and efficiency were not taken into account, and separate testing should be conducted to determine what changes could be made to the design to optimize performance. A possible future study could

take into account propagation path delays and routing efficiency when designing test cases. With that being said, this project was able to demonstrate the capabilities of Verilog to test structural-level logic and data-flow programming, while also creating a module that will be useful for future architecture integrations. Instead of needing to perform development at the physical prototyping level, which would require a large amount of resources such as discrete logic gates, wires, and chips, a design can be fully tested and implemented digitally with minimal use of physical resources.

This report was compiled using L^AT_EX .